Cours MOdélisation, Vérification et EXpérimentations
Exercices (avec les corrections)
Série A Annotation, modélisation, vérification,validation en TLA$^+$ et PlusCal avec l'outil TLC
par Dominique Méry
1er avril 2026

Table des matières

1 TD1 **2**

2 TD2 **6**

3 TD3 **14**

4 TD4 **25**

1 TD1

Exercice 1 *(malgtd1ex1)*
Le PGCD de deux nombres vérifie les propriétés suivantes :
- $\forall a, b \in \mathbb{N}.pgcd(a, b) = pgcd(b, a)$
- $\forall a, b \in \mathbb{N}.pgcd(a, a+b) = pgcd(a, b)$

mov
- *Ecrire une spécification TLA$^+$ calculant le PGCD de deux nombres donnés.*
- *Donner une explication ou une justification de la correction de cette solution*

◇– **Solution de l'exercice 1**
Modèle solution TLA$^+$:
../tlamodels/malgtd1ex1.tla

```
------------------- MODULE malgtd1ex1 -----------------
EXTENDS Naturals
----------------------------------------
CONSTANTS a,b
---------------------------------------
VARIABLES  x,y
-----------------------------------
ASSUME a \in Nat /\ b \in Nat
toto == x=a /\ y=b
-----------------------------------
(* actions *)
a1 ==
    /\ x > y
    /\ x'=x-y
    /\ y'=y
a2 == x < y /\ y'=y-x /\ x'=x
over == x=y /\ x'=x /\ y'=y
------------------------------------
go == a1 \/ a2  \/ over

-------------------------------------
(* PropriÃ©tÃ©s de sÃ»retÃ© Ã  vÃ©rifier *)
test == x # y
prop1 == x \geq 0 \* ok
prop2 == x+y \leq a+b \* ok
tocheck == prop1 /\ prop2
=============================================
```

Fin 1

Exercice 2 *(malgtd1ex2)*
L'accès à une salle est contrôlé par un système permettant d'observer les personnes qui entrent ou qui sortent de cette salle. Ce système est un ensemble de capteurs permettant d'identifier le passage d'une personne de léxtérieur vers l'intérieur et de l'intérieur à léxtérieur. Le système doit garantir qu'au plus `max` *personnes soient dans la salle. Ecrire un module TLA$^+$ permettant de modéliser un tel système respectant la propriété attendue.*

◇– **Solution de l'exercice 2**

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex2.tla

```
-------------------------------- MODULE malgtd1ex2 --------------------------------
(* modules de base importables *)
EXTENDS Naturals,TLC
---------------------------------------------------------------------------
CONSTANTS max
---------------------------------------------------------------------------
VARIABLES  np
---------------------------------------------------------------------------
(*  tentative 1 *)
entrer == np'=np +1
sortir == np'=np-1
Next ==
     \/ entrer
     \/ sortir
Init == np=0
---------------------------------------------------------------------------
(* tentative 2 *)

sortir2 == np>0 /\ np'=np-1
next2 ==  entrer \/  sortir2


entrer2 == np<max /\ np'=np+1
next3 ==  entrer2  \/  sortir2
-------------------------------------
------------------
safety1 == np \leq max \* ok
safety2 == 0 \leq np \* ok
question1 ==  np # 6
tocheck == question1
===========================================================================
```

Nous donnons trois solutions possibles selon notre analyse :

- la première solution propose deux actions *entrer* et *sortir* et on définit une relation de transition *next*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. Les deux questions produisent un échec et donc la propriété de sûreté nést pas vérifiée.
- la deuxième solution propose deux actions *entrer2* et *sortir* et on définit une relation de transition *next2*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. Il n'y a pas de retour sur la question *safety1*.
- la troisième solution propose deux actions *entrer2* et *sortir2* et on définit une relation de transition *next3*. On définit un modèle en instanciant *max* et en testant les deux propriétés de sûreté *question1* et *safety1*. L'utilisation de l'outil conduit à la vérification de la propriété de *safety1*.

Fin 2

Exercice 3 *(malgtd1ex3)*
On considère l'algorithme suivant décrit par un organigramme ou flowchart de la figure 1. Cet algorithme calcule le reste et le quotient de la division de x_1 par x_2 : $0 \leq z_2 \leq x_2 \wedge x_1 = z_1 \cdot x_2 + z_2$. On suppose que x_1 et x_2 sont positifs et non nuls.

Question 3.1 *Donner la précondition et la postcondition associées à cet algorithme.*

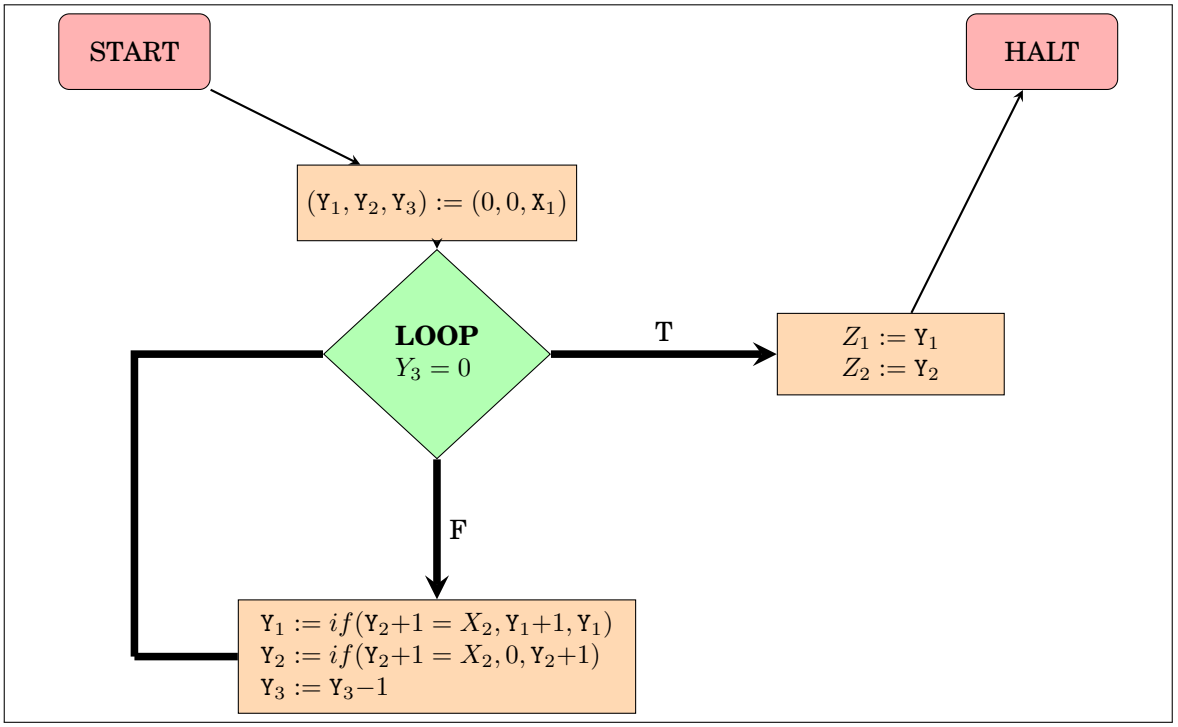

FIGURE 1 – Organigramme de calcul de la division entière

Question 3.2 *Traduire cet algorithme sous forme d'un module TLA$^+$.*

Question 3.3 *Tester les valeurs des variables à l'exécution.*

Question 3.4 *Montrer que cet algorithme est partiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer.*

◇– **Solution de l'exercice 3**

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex3.tla

```
------------------- MODULE malgtd1ex3  -----------------
EXTENDS Integers,TLC,Naturals
CONSTANTS UND,x10,x20,maxi,mini
VARIABLES x1,x2,y1,y2,y3,z1,z2,pc
----------------------------------
ASSUME x10 \in Nat /\ x20 \in Nat /\ x20 # 0
labels == {"START","LOOP","HALT"}

Init ==
    /\ pc="START"
    /\ x1 = x10 /\ x2 = x20
    /\ y1=UND /\ y2=UND /\ y3 = UND
    /\ z1=UND /\ z2 =UND

(*  y1 \in min..max  /\ y2 \in min..max /\ y3 \in min..max /\ z1 \in min..max /\ z2

start_loop ==
    /\ pc = "START"
    /\ pc' = "LOOP"
```

```
    /\ y1'=0 /\ y2'=0 /\ y3'=x1
    /\ UNCHANGED <<z1,z2,x1,x2>>

loop_loop ==
    /\ pc = "LOOP" /\ y3 # 0
    /\ y1' = IF y2+1=x2  THEN y1+1 ELSE y1
    /\ y2' = IF y2+1=x2  THEN 0 ELSE y2+1
    /\ y3' = y3 -1
    /\ UNCHANGED <<pc,x1,x2,z1,z2>>

 loop_halt ==
    /\ pc = "LOOP" /\ pc'="HALT" /\ y3 = 0
    /\ z1' = y1 /\ z2'=y2
    /\ UNCHANGED <<x1,x2,y1,y2,y3>>

 Over ==
    /\ pc="HALT"  /\ PrintT(z1) /\  PrintT(z2)
    /\ UNCHANGED<<pc,x1,x2,y1,y2,y3,z1,z2>>

  next == start_loop \/ loop_loop  \/ loop_halt\/ Over

  ------------------------
  safety1 == pc="HALT" => 0 \leq z2  /\ z2  < x2 /\ x1=z1*x2+z2 /\ x1=x10 /\ x2=x20

  D == mini..maxi

  DD(X) ==  (X # UND => X \in D)

  safety2 == DD(y1) /\ DD(y2) /\ DD(y3)  /\ DD(z1) /\ DD(z2)


  test == safety1
=============================================
```

Fin 3

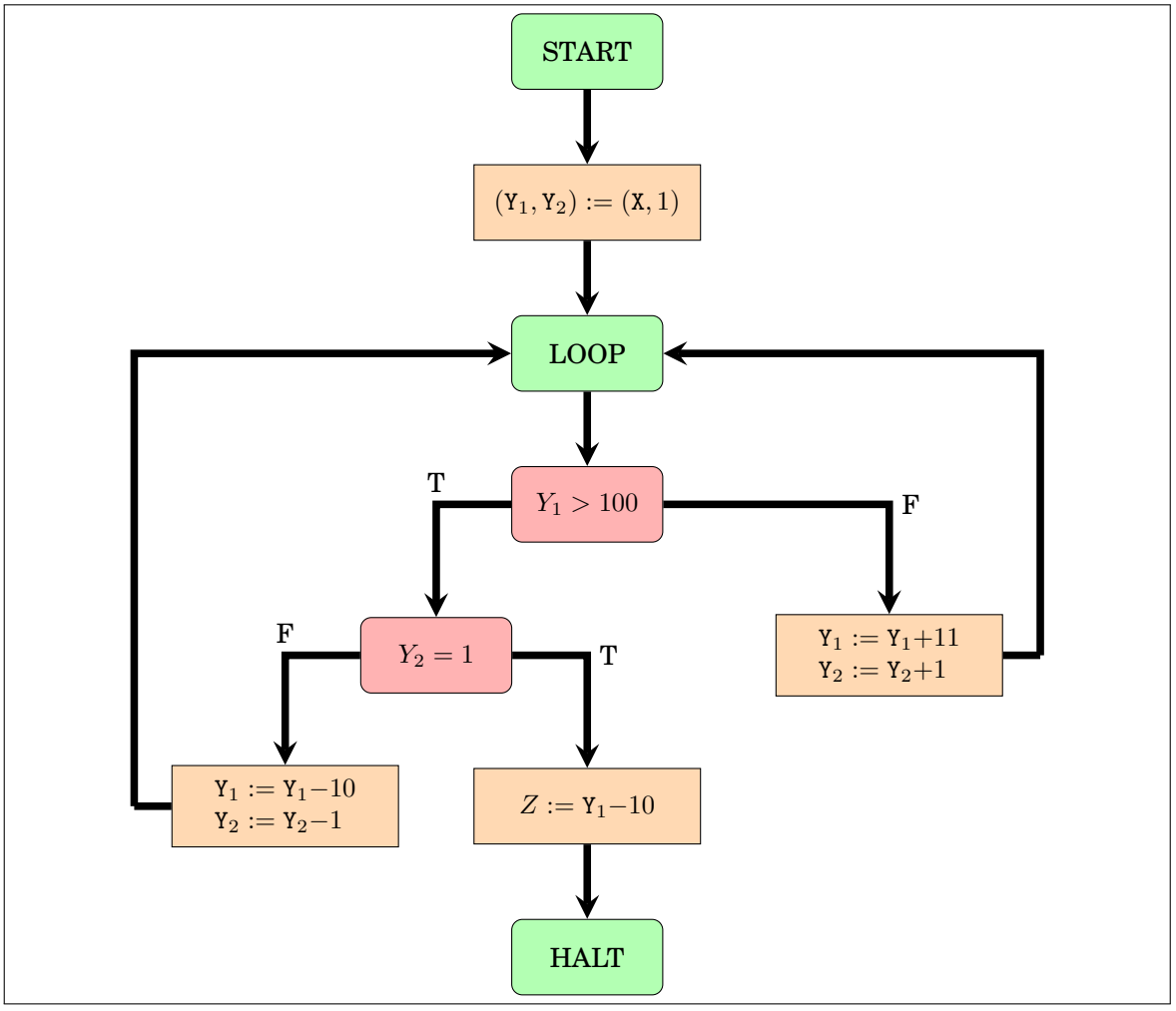

FIGURE 2 – Flowchart du calcul de la fonction de McCarthy

2 TD2

Exercice 4 *(malgtd1ex4)*
La fonction de McCarthy $f91$ *est définie pour tout entier* x $f91(x) = if\ x > 100\ then\ x{-}10\ else\ 91\ fi$.

Question 4.1 *Définir le contrat é'tablissant la correction partielle de l'algorithme* $ALG91$ *de la figure 2 qui est réputé calculer la fonction* $f91$

Question 4.2 *Construire un module TLA$^+$ modélisant les différents pas de calcul.*

Question 4.3 *Evaluer l'algorithme en posant des questions de sûreté suivantes :*

1. *l'algorithme est partiellement correct.*
2. *l'algorithme n'a pas dérreurs à léxécution.*

◇– **Solution de l'exercice 4**
Modèle solution TLA$^+$:
../tlamodels/malgtd1ex4.tla

```
-------------------------------- MODULE malgtd1ex4 --------------------------
EXTENDS Naturals,TLC,Integers
```

```
-----------------------------------------------------------------------
(* contract *)
(* variables x,y1,y2,z *)
(* requires x0 \in Nat /\ y10,y20,z0 \in Nat /\ pc="l0" *)
(* ensures zf=f91(x0) *)

-----------------------------------------------------------------------
CONSTANTS x0
-------------------------------------------------------------------
(* auxiliary definitions *)
mini  ==  -2^15
maxi == 2^15-1
D == mini..maxi
UND == -650000
f91 == [i \in Int |-> IF i > 100 THEN i-10 ELSE 91]
-----------------------------------------------------------------------
VARIABLES  x,y1,y2,z,pc
-----------------------------------------------------------------------
(* preconditions *)
ASSUME x0 \geq 0
-----------------------------
(* actions *)
a  ==
    /\ pc="START"
    /\ y1'=x /\ y2'=1
    /\ pc'="LOOP"
    /\ UNCHANGED <<x,z>>

b ==
    /\ pc="LOOP" /\  y1  \leq 100
    /\ y1'=y1+11 /\ y2'=y2+1
    /\  UNCHANGED <<x,z,pc>>

cc ==
    /\ pc="LOOP" /\  y1  > 100 /\ y2#1
    /\ y1'=y1-10 /\ y2'=y2-1
    /\  UNCHANGED <<x,z,pc>>
    /\ PrintT(y1) /\ PrintT(y2)
d ==
    /\ pc="LOOP" /\  y1  > 100 /\ y2=1
    /\ z'=y1-10 /\ pc'="HALT"
    /\ UNCHANGED <<x,y1,y2>>
-----------------------------------------------------------------------
(* spcification *)
Next == a \/ b \/ cc \/ d  \/ UNCHANGED <<y1,y2,z,x,pc>>
init1 == x=x0 /\ y1 \in Int /\ y2 \in Int /\ z \in Int /\ pc = "START"
Init  == y1 = UND  /\ y2 = UND /\ z = UND  /\  x =  x0 /\ pc = "START"
-----------------------------------------------------------------------
(* analyse *)
Q1 == pc#"HALT"  (* pc prned la valeur HALT *)  (* fausse *)
Qpc == pc="HALT" => z=IF x>100 THEN x-10 ELSE 91
Q(y) ==  y # UND => mini \leq y /\ y \leq maxi
Qover == Q(y1) /\ Q(y2) /\ Q(z) /\ Q(x)
Q2== Qpc /\ Qover
tocheck == Qover
=============================================
```

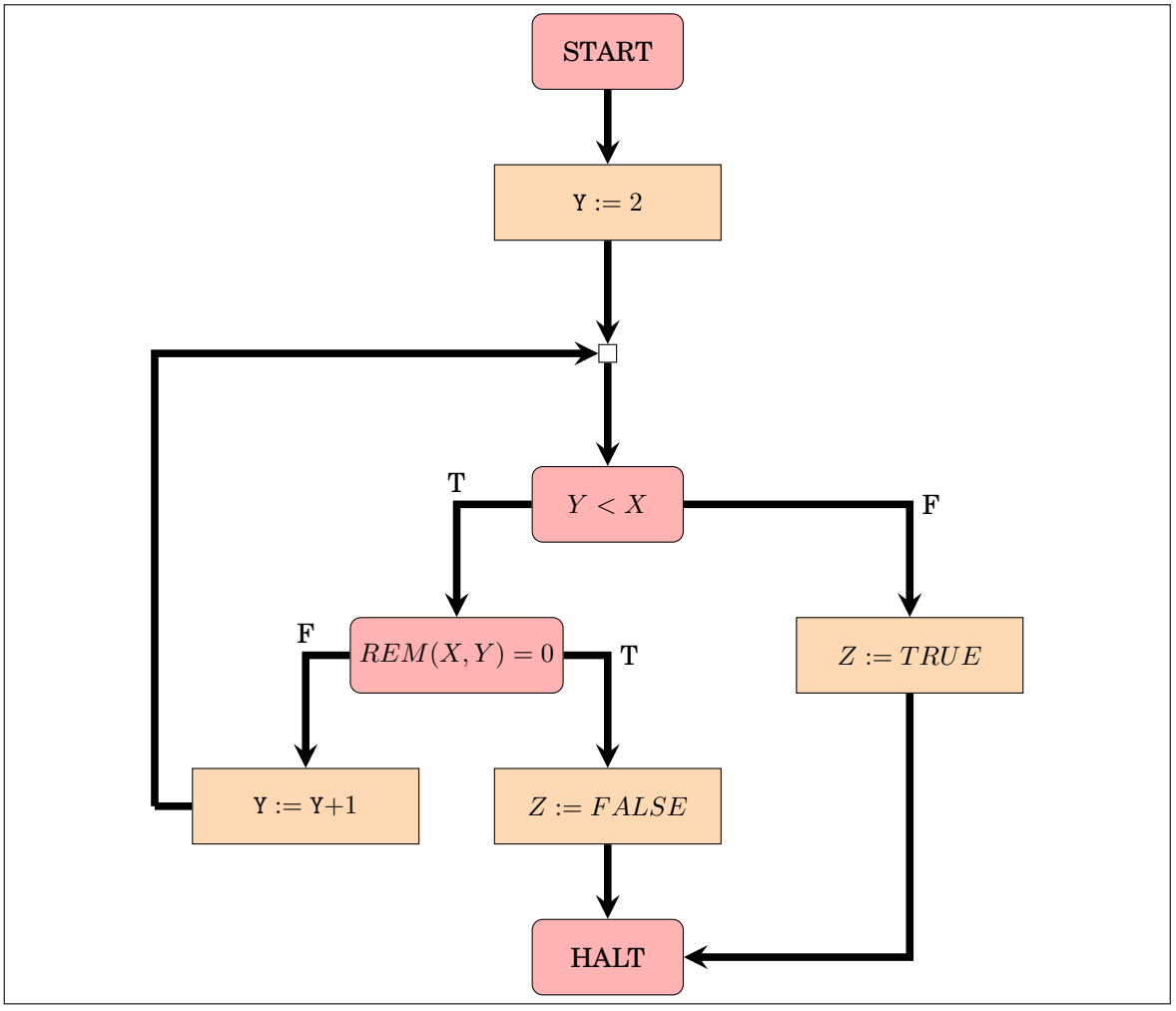

FIGURE 3 – Flowchart pour le test de primalité

Fin 4

Exercice 5 *(malgtd1ex5, malgtd1ex5bis)*
Soit le schéma de la figure 3 définissant un calcul déterminant, si un nombre entier naturel est premier ou non.

Question 5.1 *Ecrire un module TLA/TLA$^+$ modélisant ce schéma de calcul et montrer que le modèle est sans blocage.*

Question 5.2 *Définir la propriété prime(x) qui est vraie si x est premier et faux sinon.*

Question 5.3 *Ecrire le contrat présumé du calcul du flowchart de la figure 3*

requires $x0 > 0 \wedge x0, y0 \in \mathbb{Z} \wedge z0 \in \mathbb{B}$
ensures $zf = prime(x0) \wedge xf = x0$

Question 5.4 *Enoncer et vérifier la correction partielle*

Question 5.5 *Enoncer et vérifier l'absence dérreurs à léxécution.*

◇– Solution de l'exercice 5

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex5.tla

```
------------MODULE malgtd1ex5 --------

EXTENDS Integers,TLC
-----------------------------------------------------------
---------------------------------------------------------------------------
(* contract *)
(* variables x,y,z *)
(* requires x0 \in Nat /\ y0 \in Nat /\ Z \IN BOOL *)
(* ensures zf= prime(x0) *)
CONSTANTS mini,maxi,und, bund (* constants for undefinedness,  bounds of domain *)
-----------------------------------------------------------
(* requires *)
CONSTANTS x0  (* x0 is the input *)
-----------------------------------------------------------
(* precondition *)
ASSUME x0 \in Nat
-----------------------------------------------------------
VARIABLES  x,y,z,pc
-----------------------------------------------------------
Init ==  x = x0 /\ y=und  /\ z=bund /\ pc="start"
-----------------------------------------------------------
L1 == pc = "start" /\ y'=2 /\ pc'="loop" /\ UNCHANGED <<x,z>>
L2 == pc = "loop" /\ y \geq  x /\ z'=TRUE /\ pc'="halt" /\ UNCHANGED << x,y>>
L3 == pc="loop" /\ y<x /\ x % y =0 /\ z'= FALSE /\ pc'="halt" /\ UNCHANGED <<x,y>>
L4 == pc="loop" /\ y<x /\ x % y # 0  /\ y'=y+1 /\ UNCHANGED << pc,x,z>>
skip == UNCHANGED << pc,x,z,y  >>
-----------------------------------------------------------
Next == L1 \/ L2 \/ L3 \/ L4 \/ skip
----------------------------------------------------------
(* auxiliary definitions *)
prime(u) == \A v \in 2..u-1: u % v # 0   (* define that u is a prime number *)
Dbool == {FALSE,TRUE}
Dint == mini..maxi (* domain for integer variables *)
DDint(v) == v # und => v \in Dint
DDbool(v) == v # bund => v \in Dbool
----------------------------------------------------------
(* properties to check *)
SafePC == pc="halt" => z=prime(x0) /\ PrintT(z)(* the algorithm is partially  corre
SafeRTE == DDint(y) /\ DDbool(z) (* the  algorithm is  runtime errors free. *)
Safe == SafePC /\ SafeRTE
=============================================================================
```

Modèle solution TLA$^+$:
../tlamodels/inmalgtd1ex5.tla

```
------------MODULE inmalgtd1ex5 --------

EXTENDS Integers,TLC
-----------------------------------------------------------
CONSTANTS mini,maxi,und, bund (* constants for undefinedness,  bounds of domain *)
```

```
-----------------------------------------------------------
prime(u) == \A v \in 2..u-1: u % v # 0   (* define that x is a prime number *)
(* requires *)
CONSTANTS x0,y0,z0  (* x is the input *)
ASSUME x0 \in Nat /\ y0 \in Int /\ z0 \in {FALSE,TRUE}
(* ensures *)
post(u0,v0,w0,u,v,w) == (w = prime(u0))
-----------------------------------------------------------
VARIABLES  x,y,z,pc
-----------------------------------------------------------
Init ==  x=x0 /\ y=y0  /\ z=z0 /\ pc="start"
-----------------------------------------------------------
L1 == pc = "start" /\ y'=2 /\ pc'="loop" /\ UNCHANGED <<x,  z >> /\ PrintT(y)
L2 == pc = "loop" /\ y \geq  x /\ z'=TRUE /\ pc'="halt" /\ UNCHANGED << x,y  >>
L3 == pc="loop" /\ y<x /\ x % y =0 /\ z'= FALSE /\ pc'="halt" /\ UNCHANGED << x,y
L4 == pc="loop" /\ y<x /\ x % y # 0  /\ y'=y+1 /\ UNCHANGED << x,pc,z  >>
skip == UNCHANGED << pc,z,y,x >>
-----------------------------------------------------------
Next == L1 \/ L2 \/ L3 \/ L4 \/ skip
-----------------------------------------------------------
(* auxiliary definitions *)
Dint == mini..maxi (* domain for integer variables *)
Dbool == {FALSE,TRUE}
DDint(v) == v # und => v \in Dint
DDbool(v) == v # bund => v \in Dbool
-----------------------------------------------------------
Q1 == pc="halt" => post(x0,y0,z0,x,y,z) (* is the  algorithm partially correct? *)
SafePC == pc="halt" => post(x0,y0,z0,x,y,z) (* the algorithm is partially  correct
Q2 == pc # "halt"
Q3 == DDint(x) /\ DDint(y) /\ DDbool(z) (* is the  algorithm runtime errors free? *
SafeRTE == DDint (x) /\ DDint(y) /\ DDbool(z) (* the  algorithm is  runtime errors
Safe == SafePC /\ SafeRTE
================
```

Fin 5

Exercice 6 *(malgtd1ex11,pluscal_max.tla)*

Question 6.1 *Ecrire un module TLA$^+$ qui traduit la relation de transition de cet algorithme (Algorithme 6) selon les instructions. Pour cela, vous utiliserez la fonctionnalité offerte par la tradution d'un algorithme PlusCal en TLA$^+$. La figure est intitulée maximum de deux nombres non annotée.*

Question 6.2 *Compléter l'algorithme intitulé maximum de deux nombres non annotée en l'annotant.*

◇– **Solution de la question 6.2**

L'annotation de cet algorithme est donnée à la référence d'algorithme maximum de deux nombres annotée et la figure est placée au gré de LaTeXsous le numéro 6 .

Fin 6.2

Question 6.3 *Vérifier que l'annotation est correcte.*

◇– **Solution de la question 6.3**

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex11.tla

Variables : X,Y,Z
Requires : $x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}$
Ensures : $z_f = max(x_0, y_0)$

$\ell_0 : \{\ldots\}$
if $X < Y$ **then**
  $\ell_1 : \{\ldots\}$
  $Z := Y;$
  $\ell_2 : \{\ldots\}$
else
  $\ell_3 : \{\ldots\}$
  $Z := X;$
  $\ell_4 : \{\ldots\}$
;
$\ell_5 : \{\ldots\}$

Algorithme 1: maximum de deux nombres non annotée

Variables : X,Y,Z
Requires : $x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}$
Ensures : $z_f = max(x_0, y_0)$

$\ell_0 : \{x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}\}$
if $X < Y$ **then**
  $\ell_1 : \{x < y \wedge x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}\}$
  $Z := Y;$
  $\ell_2 : \{x < y \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z} \wedge z = y_0\}$
else
  $\ell_3 : \{x \geq y \wedge x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}\}$
  $Z := X;$
  $\ell_4 : \{x \geq y \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z} \wedge z = x_0\}$
;
$\ell_5 : \{z = max(x_0, y_0) \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \ \wedge z_0 \in \mathbb{Z}\}$

Algorithme 2: maximum de deux nombres non annotée

Variables : X,Y,Z
Requires : $x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z}$
Ensures : $z_f = max(x_0, y_0)$

$\ell_0 : \{x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z}\}$
if $X < Y$ **then**
  $\ell_1 : \{x < y \wedge x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z}\}$
  $Z := Y;$
  $\ell_2 : \{x < y \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z} \wedge z = y_0\}$
else
  $\ell_3 : \{x \geq y \wedge x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z}\}$
  $Z := X;$
  $\ell_4 : \{x \geq y \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z} \wedge z = x_0\}$
;
$\ell_5 : \{z = max(x_0, y_0) \wedge x = x_0 \wedge y = y_0 \wedge x_0, y_0 \in \mathbb{N} \wedge z_0 \in \mathbb{Z}\}$

Algorithme 3: maximum de deux nombres non annotée

```
------------------------------- MODULE malgtd1ex11 -------------------------------
EXTENDS Naturals, Integers
CONSTANTS x0,y0,z0,mini0,maxi0
VARIABLES  x,y,z,pc
typeInt(u) == u \in Int
maxi(u,v) == IF u < v THEN v ELSE u
pre ==  x0 \in Nat /\ y0 \in Nat /\ z0 \in Int
ASSUME pre
---------------------------------------------------------------------
al0l1 ==
    /\ pc="l0"
    /\ pc'="l1"
    /\ x<y
    /\ z'=z /\ x'=x /\ y'=y
al1l2 ==
    /\ pc="l1"
    /\ pc'="l2"
    /\ z'=y
    /\ x'=x /\ y'=y
al2l5 ==
    /\ pc="l2"
    /\ pc'="l5"
    /\ z'=z /\ x'=x /\ y'=y
al0l3 ==
    /\ pc="l0"
    /\ pc'="l3"
    /\ x \geq y
    /\ UNCHANGED <<z,x,y>>
al3l4 ==
    /\ pc="l3"
    /\ pc'="l4"
    /\ z'=x
    /\ x'=x /\ y'=y
  al4l5 ==
```

```
    /\ pc="l4"
    /\ pc'="l5"
    /\ z'=z /\ x'=x /\ y'=y
---------------------
Next == al0l1 \/ al1l2 \/ al2l5  \/ al0l3 \/ al3l4 \/ al4l5 \/ UNCHANGED <<x,y,z,pc
Init == pc="l0" /\ x=x0 /\ y =y0 /\ z = z0
---------------------
i ==
    /\ typeInt(x) /\ typeInt(y) /\ typeInt(z)
    /\ pc="l0" =>  x=x0 /\ y=y0 /\ z=z0 /\ pre
    /\ pc="l1" => x<y /\ x=x0 /\ y=y0 /\ z=z0 /\ pre
    /\ pc="l2" => x<y /\ x=x0 /\ y=y0 /\ z=y0 /\ pre
    /\ pc="l3" => x \geq y /\ x=x0 /\ y=y0 /\ z=z0 /\ pre
    /\ pc="l4" => x \geq y /\ x=x0 /\ y=y0 /\ z=x0  /\ pre
    /\ pc="l5" => z = maxi(x0,y0) /\ x=x0 /\ y=y0  /\ pre
safepc ==  pc="l5" =>  z = maxi(x0,y0)
safeab == x=x0 /\ y=y0
saferte ==
    /\ mini0 <= x /\ x <= maxi0
    /\ mini0 <= y /\ y <= maxi0
==============================================================================
\* Modification History
\* Last modified Wed Feb 08 10:52:27 CET 2023 by mery
\* Created Wed Sep 09 18:19:08 CEST 2015 by mery
```

Fin 6.3

Question 6.4 *Compléter le module TLA$^+$ en définissant l'invariant construit avec les annotations et vérifier le contrat.*

3 TD3

Exercice 7 *Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA$^+$ pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA$^+$ contenant une expression de l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence dérreurs à léxécution. Cet exercice ressemble aux exercices précédents mais se focalise sur le langage algorithmique PlusCal qui est traduit automatiquement comme cela a été fait manuellement.*

Question 7.1 *(appex4_1_1)*
Soit l'annotation suivante :

$\ell_1 : x = 10 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
$y := z{+}x$
$\ell_2 : x = 10 \ \wedge \ y = x{+}2{\cdot}10$

Traduire en PlusCal.

Listing 1 – appex4_1_1.tla

```
--------- MODULE appex4_1_1 --------
EXTENDS Integers,TLC
-----------------------------------------------------------------
CONSTANTS x0,y0,z0
-----------------------------------------------------------------
pre(a,b,c)  == a=10 /\ c=2*a /\ b=c+a
-----------------------------------------------------------------
(* Verifying the existence of a triple that satisfies pre(x0;y0,z0) *)
ASSUME pre(x0,y0,z0)
-----------------------------------------------------------------
(*
--algorithm  test  {
variables x=x0,z=z0,y=y0,t;
{
l1:\* assert x=10 /\ z=2*x /\ y = z + x;
y:=z+x;
l2: \* assert x = 10 /\ y = x + 2*10
    skip ;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "cec661f" /\ chksum(tla) = "51e99e7b")
CONSTANT defaultInitValue
VARIABLES x, z, y, t, pc

vars == << x, z, y, t, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ z = z0
        /\ y = y0
        /\ t = defaultInitValue
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ y' = z+x
      /\ pc' = "l2"
```

```
        /\ UNCHANGED << x, z, t >>

l2 == /\ pc = "l2"
      /\ TRUE
      /\ pc' = "Done"
      /\ UNCHANGED << x, z, y, t >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == l1 \/ l2
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

\* ASSUME pre
-----------------------------------------------------------------
(* premiminaries for checking  on  pssible values *)
MAX == 32767  (* 16 bits *)
D == -32768..32767
DD(X) ==  (X #defaultInitValue) => (X \in D)
-----------------------------------------------------------------
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int /\ z \in Int
    /\ pc="l1" =>  x=10 /\  z=2*x /\ y=z+x
    /\ pc="l2" =>   x=10 /\ y=x+2*10
    /\ pc="Done" =>   x=10 /\ y=x+2*10
Safety_Partial_Correctness == pc="Done" =>   x=10 /\ y=x+2*10
Safety_rte ==  DD(x)  /\ DD(y) /\  DD(z) /\ DD(t)
check == inv /\ Safety_Partial_Correctness /\ Safety_rte
prop == [] (x=x0)
=================================================
```

◇– **Solution de la question 7.1**

Fin 7.1

Question 7.2 *(appex4_1_2)*

On suppose que p *est un nombre premier :*

$\ell_1 : x = 2^p \ \wedge\ y = 2^{p+1} \ \wedge\ x{\cdot}y = 2^{2\cdot p+1}$
$x := y{+}x{+}2^x$
$\ell_2 : x = 5{\cdot}2^p \ \wedge\ y = 2^{p+1}$

```
--------- MODULE appex4_1_2 --------
EXTENDS Integers, Naturals,TLC
CONSTANTS x0,y0,p,maxi
-----------------------------------------------------------------
diviseurs(n) == { m \in 1..maxi:  n % m = 0 }
premier(n) ==  diviseurs(n) = {1,n} /\ n # 1
pre == x0=2^p /\ y0=2^(p+1) /\ x0*y0=2^(2*p+1)
-----------------------------------------------------------------
```

```
ASSUME premier(p) /\ pre
(*

--algorithm test {
variables x=x0,y=y0;
{
l1:assert x=2^p /\ y=(2^p)*2 /\ x*y=2^(2*p+1);
x:=y+x+(2^x);
l2:assert x=5*(2^p) /\ y=2^(p+1) ;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "31a0954b" /\ chksum(tla) = "e1037380")
VARIABLES x, y, pc

vars == << x, y, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ y = y0
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=2^p /\ y=(2^p)*2 /\ x*y=2^(2*p+1),
                "Failure␣of␣assertion␣at␣line␣16,␣column␣4.")
      /\ x'␣=␣y+x+(2^x)
␣␣␣␣␣␣/\␣pc' = "l2"
      /\ y'␣=␣y

l2␣==␣/\␣pc␣=␣"l2"
␣␣␣␣␣␣/\␣Assert(x=5*(2^p)␣/\␣y=2^(p+1),
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"Failure␣of␣assertion␣at␣line␣18,␣column␣4.")
␣␣␣␣␣␣/\␣pc' = "Done"
      /\ UNCHANGED << x, y >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars

Next == l1 \/ l2
           \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

---------------------------------------------------------------
MAX == 32768  (* 16 bits *)
D == 0..32768
DD(X) == ( X \in D)
DDP(X) ==  (X # defaultInitValue) => (X \in D)
Safety_absence ==  DD(x)  /\ DD(y)
Inv ==
    /\ pc \in {"l1","l2","Done"}
    /\ x \in Int /\ y \in Int
```

```
    /\ pc="l1" => x=2^p /\ y=2^p*2 /\  x*y=2^(2*p+1)
    /\ pc="l2" =>     x=5*2^p /\ y=2^(p+1)

==========================================
```

Question 7.3 *(appex4_1_3)*

$\ell_1 : x = 1 \ \wedge \ y = 12$
$x := 2{\cdot}y$
$\ell_2 : x = 1 \ \wedge \ y = 24$

```
--------- MODULE appex4_1_3 --------
EXTENDS Integers,TLC
-----------------------------------------------------------------
CONSTANTS x0,y0
-----------------------------------------------------------------
pre == x0=1 /\ y0=12

-----------------------------------------------------------------
(*

--algorithm test {
variables x=x0,y=y0;
{
l1:assert x=1 /\ y=12;
x:=2*y;
l2:assert x=24 /\ y=12;
}
}
*)
\* BEGIN TRANSLATION (chksum(pcal) = "fdb997c5" /\ chksum(tla) = "97306b1b")
VARIABLES x, y, pc

vars == << x, y, pc >>

Init == (* Global variables *)
        /\ x = x0
        /\ y = y0
        /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=1 /\ y=12, "Failure␣of␣assertion␣at␣line␣14,␣column␣4.")
      /\ x'␣=␣2*y
␣␣␣␣␣␣/\␣pc' = "l2"
      /\ y'␣=␣y

l2␣==␣/\␣pc␣=␣"l2"
␣␣␣␣␣␣/\␣Assert(x=24␣/\␣y=12,␣"Failure␣of␣assertion␣at␣line␣16,␣column␣4.")
␣␣␣␣␣␣/\␣pc' = "Done"
      /\ UNCHANGED << x, y >>

(* Allow infinite stuttering to prevent deadlock on termination. *)
Terminating == pc = "Done" /\ UNCHANGED vars
```

```
Next == l1 \/ l2
            \/ Terminating

Spec == Init /\ [][Next]_vars

Termination == <>(pc = "Done")

\* END TRANSLATION

-----------------------------------------------------------------
MAX == 32768  (* 16 bits *)
D == 0..32768
DD(X) == ( X \in D)
DDP(X) ==  (X # defaultInitValue) => (X \in D)
-----------------------------------------------------------------
Safety_absence ==  DD(x)  /\ DD(y)
Inv ==
     /\ pc \in {"l1","l2","Done"}
     /\ x \in Int /\ y \in Int
     /\ pc="l1" =>  x=1 /\ y=12
     /\ pc="l2" =>  x=1 /\ y=24

=============================================
```

Question 7.4 *(appex4_1_4)*

$\ell_1 : x = 11 \ \wedge \ y = 13$
$z := x; x := y; y := z;$
$\ell_2 : x = 26/2 \ \wedge \ y = 33/3$

```
--------- MODULE appex4_1_4 --------
EXTENDS Integers,TLC
-----------------------------------------------------------------

(*
--algorithm  test  {
variables x=11,y=13,z;
{
l1:assert x=11 /\ y=13;
z:=x;x:=y;y:=z;
l2:assert x=26 \div 2 /\ y=33 \div 3;

l3:print <<x,y>>;
}
}
*)
\* BEGIN TRANSLATION
CONSTANT defaultInitValue
VARIABLES x, y, z, pc

vars == << x, y, z, pc >>

Init == (* Global variables *)
        /\ x = 11
```

```
          /\ y = 13
          /\ z = defaultInitValue
          /\ pc = "l1"

l1 == /\ pc = "l1"
      /\ Assert(x=11 /\ y=13, "Failure␣of␣assertion␣at␣line␣10,␣column␣4.")
      /\ z'␣=␣x
␣␣␣␣␣␣/\␣x' = y
      /\ y'␣=␣z'
      /\ pc'␣=␣"l2"

l2␣==␣/\␣pc␣=␣"l2"
␣␣␣␣␣␣/\␣Assert(x=26␣\div␣2␣/\␣y=33␣\div␣3,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"Failure␣of␣assertion␣at␣line␣12,␣column␣4.")
␣␣␣␣␣␣/\␣pc' = "l3"
      /\ UNCHANGED << x, y, z >>

l3 == /\ pc = "l3"
      /\ PrintT(<<x,y>>)
      /\ pc'␣=␣"Done"
␣␣␣␣␣␣/\␣UNCHANGED␣<<␣x,␣y,␣z␣>>

(*␣Allow␣infinite␣stuttering␣to␣prevent␣deadlock␣on␣termination.␣*)
Terminating␣==␣pc␣=␣"Done"␣/\␣UNCHANGED␣vars

Next␣==␣l1␣\/␣l2␣\/␣l3
␣␣␣␣␣␣␣␣␣␣␣\/␣Terminating

Spec␣==␣Init␣/\␣[][Next]_vars

Termination␣==␣<>(pc␣=␣"Done")

\*␣END␣TRANSLATION

----------------------------------------------------------------
MAX␣==␣32768␣␣(*␣16␣bits␣*)
D␣==␣0..32768
DD(X)␣==␣(␣X␣\in␣D)
DDP(X)␣==␣␣(X␣#␣defaultInitValue)␣=>␣(X␣\in␣D)
----------------------------------------
Safety_absence␣==␣␣DD(x)␣␣/\␣DD(y)
Inv␣==
␣␣␣␣/\␣pc␣\in␣{"l1","l2","Done"}
␣␣␣␣/\␣x␣\in␣Int␣/\␣y␣\in␣Int
␣␣␣␣/\␣pc="l1"␣=>␣␣x=11␣/\␣y=13
␣␣␣␣/\␣pc="l2"␣=>␣␣x=26␣\div␣2␣/\␣y=33␣\div␣3

=========
```

Exercice 8 *(tla_squareroot, pluscal_squareroot)*
On considère l'algorithme $squareroot$ *calculant la racine carrée entière d'un nombre naturel* $x \in \mathbb{N}$.

VARIABLES $X, Y1, Y2, Y3, Z$

$pre(x0, y10, y20, y30, z0) \stackrel{def}{=}$
$U \stackrel{def}{=} (\ X, Y1, Y2, Y3, Z\ )$
$u0 \stackrel{def}{=} (\ x0, y10, y20, y30, z0\ )$
$post(x0, y10, y20, y30, z0, xf, y1f, y2f, y3f, zf) \stackrel{def}{=}$

REQUIRES $pre(x0, y10, y20, y30, z0)$
ENSURES $post(x0, y10, y20, y30, z0, xf, y1f, y2f, y3f, zf)$

$\ell_0 : pre(u0) \wedge u = u0$
$(Y1, Y2, Y3) := (0, 1, 1)$
$\ell_1 : pre(u0) \wedge x = x0 \wedge z = z0 \wedge y2 = (y1{+}1){\cdot}(y1{+}1) \wedge y3 = 2{\cdot}y1{+}1 \wedge y1{\cdot}y1 \leq x$
WHILE $Y2 \leq X$ *DO*
ℓ_2 :
$(Y1, Y2, Y3) := (Y1{+}1, Y2{+}Y3{+}2, Y3{+}2);$
ℓ_3 : *OD*;
ℓ_4 :
$Z := Y1;$
ℓ_5 :

Listing 2 – mainsquareroot.c

```
#include <stdio.h>
#include <limits.h>
int spower2(int x)
{int  z,y1,y2,y3,r;
  r=0;y1=0;y2=1;y3=1;
   while (y2<= x)
     {
         y1=y1+1;
       y2= y2 + y3 + 2;
       y3=y3+2;

     }
  r=y1;
  return(r);
}

int main () {

  int counter;
    for( counter=0; counter<5; counter++ ) {
      int v,r;
      printf("Enter␣a␣natural␣number:");
      scanf("%d", &v);
      r = spower2(v);
      printf ("Power␣2:␣%d␣---->␣␣%d\n", v,r);

    };
}
```

Question 8.1 *Définir les deux assertions pre et post qui établissent le contrat de cet algorithme.*

Question 8.2 *Annoter l'algorithme et le traduire en PlusCal. On pourra faire deux types d'annotations :*

- *soit dans les instructions* `assert`.
- *soit sous la forme d'un invariant global.*

Question 8.3 *Vérifier la correction partielle et l'absence d'erreurs à l'exécution de cet algroithme en utilisant la traduction en TLA$^+$.*

L'algorithme annoté est décrit par l'algorithme **??**

VARIABLES $X, Y1, Y2, Y3, Z$

$pre(x0, y10, y20, y30, z0) \stackrel{def}{=} \begin{cases} x0 \in \mathbb{N} \wedge x0 \neq 0 \\ y10, y20, y30, z0 \in \mathbb{Z} \end{cases}$

$U \stackrel{def}{=} (\ X, Y1, Y2, Y3, Z\ )$

$u0 \stackrel{def}{=} (\ x0, y10, y20, y30, z0\ )$

$post(x0, y10, y20, y30, z0, xf, y1f, y2f, y3f, zf) \stackrel{def}{=} (\ zf^2 \leq x0 \wedge x0 \leq (zf{+}1)^2$

$pre(u0) \stackrel{def}{=} pre(x0, y10, y20, y30, z0)$

$(u = u0) \stackrel{def}{=} x = x0 \wedge y1 = y10 \wedge y1 = y10 \wedge y2 = y20 \wedge y3 = y30 \wedge z = z0$

REQUIRES $pre(x0, y10, y20, y30, z0)$
ENSURES $post(x0, y10, y20, y30, z0, xf, y1f, y2f, y3f, zf)$

$\ell_0 : pre(u0) \wedge u = u0$
$(Y1, Y2, Y3) := (0, 1, 1)$
$\ell_1 : pre(u0) \wedge x = x0 \wedge z = z0 \wedge y2 = (y1{+}1){\cdot}(y1{+}1) \wedge y3 = 2{\cdot}y1{+}1 \wedge y1{\cdot}y1 \leq x$
WHILE $Y2 \leq X$ DO
$\ell_2 : P_{\ell_1}(u0, u) \wedge y2 \leq x$
  $(Y1, Y2, Y3) := (Y1{+}1, Y2{+}Y3{+}2, Y3{+}2);$
$\ell_3 : P_{\ell_1}(u0, u)$
OD;
$\ell_4 : P_{\ell_1}(u0, u) \wedge y2 > x$
$Z := Y1;$
$\ell_5 : pre(u0) \wedge x = x0 \wedge y2 = (y1{+}1){\cdot}(y1{+}1) \wedge y3 = 2{\cdot}y1{+}1 \wedge y1{\cdot}y1 \leq x \wedge x < y2$

```
--------- MODULE tla_squareroot --------
EXTENDS Integers,TLC
CONSTANTS x,U (* x is the input *)
VARIABLES  pc,y1,y2,y3,z
-------------------------------------------------------------
vars == <<pc,y1,y2,y3,z>>
-------------------------------------------------------------
al0l1 == pc="l0" /\ y1'=0 /\y2'=1 /\ y3'=1 /\ pc'="l1" /\ z'=z /\ x'=x
al1l2 == pc="l1" /\ y2 \leq x /\pc'="l2" /\  UNCHANGED <<y1,y2,y3,z,x>>
al1l4 == pc="l1" /\ y2 >  x /\ pc'="l4" /\ UNCHANGED <<y1,y2,y3,z,x>>
al2l3 == pc="l2" /\ y1'=y1+1 /\y2'=y2+y3+2 /\ y3'=y3+2 /\ pc'="l3" /\ z'=z /\ x'=x
al3l2 == pc="l3" /\ y2 \leq x /\pc'="l2" /\  UNCHANGED <<y1,y2,y3,z,x>>
al3l4 == pc="l3" /\ y2 >  x /\ pc'="l4" /\ UNCHANGED <<y1,y2,y3,z,x>>
al4l5 == pc="l4" /\ z'=y1 /\pc'="l5" /\  UNCHANGED <<y1,y2,y3,x>>
-------------------------------------------------------------
Init == y1=U /\ y2 = U /\ y3 = U /\ z=U /\ pc="l0"
Next == al0l1 \/ al1l2 \/ al1l4 \/ al2l3 \/ al3l2 \/ al3l4 \/ al4l5
-------------------------------------------------------------
MAX == 32767  (* 16 bits *)
```

```
MIN == -32768
D == MIN..MAX
(*  x \leq 32760 *)
Safety_absence ==  (y1 \in D)  /\ (y2 \in D) /\ (y3  \in D) /\ (z \in D)
i ==
        /\ pc="l0" => y1 \in D  /\  y2 \in D /\  y3 \in D /\  z \in D
        /\ pc="l1" => y2 = (y1+1)*(y1+1) /\ y3=2*y1+1  /\  y1*y1\leq x
        /\ pc="l2" => y2 = (y1+1)*(y1+1) /\ y3=2*y1+1  /\  y1*y1\leq x /\ y2 \leq x
        /\ pc="l3" => y2 = (y1+1)*(y1+1) /\ y3=2*y1+1  /\  y1*y1\leq x
        /\ pc="l4" => y2 = (y1+1)*(y1+1) /\ y3=2*y1+1  /\  y1*y1\leq x /\ x < y2
        /\ pc="l5" => y2 = (y1+1)*(y1+1) /\ y3=2*y1+1  /\  z*z\leq x /\ x < (z+1)*(
Safety_partialcorrectness == pc="l5" => z*z\leq x /\ x < (z+1)*(z+1)

Qtermination == pc # "l5"

=========

--------- MODULE pluscal_squareroot --------
EXTENDS Integers,TLC
CONSTANTS x  (* x is the input *)
----------------------------------------------------------------

(*
--algorithm  squareroot {
variables y1,y2,y3,z;
{
l0: assert (y1 = defaultInitValue /\ y2 = defaultInitValue /\ y3 = defaultInitValue
y1:=0;y2:=1;y3:=1;
l1: assert y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1  \leq x;
l:while (y2 \leq x){
    l2: assert y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y2  \leq x;
    y1:=y1+1;
    y2:=y2+y3+2; y3:=y3+2;
    l3: assert y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1  \leq x;

    };
l4: assert y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1  \leq x /\ x < y2;
z:=y1;
l5: print <<x,z,y1,y2,y3>>;
    }
 }

*)
\* BEGIN TRANSLATION (chksum(pcal) = "d6c5dbb6" /\ chksum(tla) = "48b1ccaa")
CONSTANT defaultInitValue
VARIABLES y1, y2, y3, z, pc

vars == << y1, y2, y3, z, pc >>

Init == (* Global variables *)
        /\ y1 = defaultInitValue
        /\ y2 = defaultInitValue
        /\ y3 = defaultInitValue
        /\ z = defaultInitValue
        /\ pc = "l0"
```

```
l0 == /\ pc = "l0"
      /\ Assert((y1 = defaultInitValue /\ y2 = defaultInitValue /\ y3 = defaultInitV
                 "Failure␣of␣assertion␣at␣line␣10,␣column␣5.")
      /\ y1'␣=␣0
␣␣␣␣␣␣/\␣y2' = 1
      /\ y3'␣=␣1
␣␣␣␣␣␣/\␣pc' = "l1"
      /\ z'␣=␣z

l1␣==␣/\␣pc␣=␣"l1"
␣␣␣␣␣␣/\␣Assert(y2␣=␣(y1+1)*(y1+1)␣/\␣y3␣=␣2*y1+1␣/\␣y1*y1␣␣\leq␣x,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"Failure␣of␣assertion␣at␣line␣12,␣column␣5.")
␣␣␣␣␣␣/\␣pc' = "l"
      /\ UNCHANGED << y1, y2, y3, z >>

l == /\ pc = "l"
     /\ IF y2 \leq x
           THEN /\ pc'␣=␣"l2"
␣␣␣␣␣␣␣␣␣␣␣ELSE␣/\␣pc' = "l4"
     /\ UNCHANGED << y1, y2, y3, z >>

l2 == /\ pc = "l2"
      /\ Assert(y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y2  \leq x,
                 "Failure␣of␣assertion␣at␣line␣14,␣column␣9.")
      /\ y1'␣=␣y1+1
␣␣␣␣␣␣/\␣y2' = y2+y3+2
      /\ y3'␣=␣y3+2
␣␣␣␣␣␣/\␣pc' = "l3"
      /\ z'␣=␣z

l3␣==␣/\␣pc␣=␣"l3"
␣␣␣␣␣␣/\␣Assert(y2␣=␣(y1+1)*(y1+1)␣/\␣y3␣=␣2*y1+1␣/\␣y1*y1␣␣\leq␣x,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"Failure␣of␣assertion␣at␣line␣17,␣column␣9.")
␣␣␣␣␣␣/\␣pc' = "l"
      /\ UNCHANGED << y1, y2, y3, z >>

l4 == /\ pc = "l4"
      /\ Assert(y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1  \leq x /\ x < y2,
                 "Failure␣of␣assertion␣at␣line␣20,␣column␣5.")
      /\ z'␣=␣y1
␣␣␣␣␣␣/\␣pc' = "l5"
      /\ UNCHANGED << y1, y2, y3 >>

l5 == /\ pc = "l5"
      /\ PrintT(<<x,z,y1,y2,y3>>)
      /\ pc'␣=␣"Done"
␣␣␣␣␣␣/\␣UNCHANGED␣<<␣y1,␣y2,␣y3,␣z␣>>

(*␣Allow␣infinite␣stuttering␣to␣prevent␣deadlock␣on␣termination.␣*)
Terminating␣==␣pc␣=␣"Done"␣/\␣UNCHANGED␣vars

Next␣==␣l0␣\/␣l1␣\/␣l␣\/␣l2␣\/␣l3␣\/␣l4␣\/␣l5
␣␣␣␣␣␣␣␣␣␣␣\/␣Terminating

Spec␣==␣Init␣/\␣[][Next]_vars
```

```
Termination ==  <>(pc = "Done")

\* END TRANSLATION


MAX == 32768   (* 16 bits *)
D == 0..MAX
(*  x \leq 32768 *)
DD(X) ==   ( X # defaultInitValue =>  X \in D)
init(X) ==   ( X # defaultInitValue)

Safety_absence ==  DD(y1)   /\ DD(y2) /\ DD(y3) /\ DD(z)

Safety_partialcorrectness == pc="Done" =>   z*z\leq x /\ x < (z+1)*(z+1)

Inv ==
    /\ pc \in {"l","l0","l1","l2","l3","l4","Done","l5","l33"}
    /\   (init(y1) /\ init(y2) /\ init(y3) /\ init(z)) => (y1 \in Nat /\ y2 \in Nat
    /\ pc = "l0" =>   (y1 = defaultInitValue /\ y2 = defaultInitValue /\ y3 = defaul
    /\ pc = "l1"   => y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1   \leq x
    /\ pc = "l2" => y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y2   \leq x
    /\ pc="l3" => y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1   \leq x
    /\ pc="l4" => y2 = (y1+1)*(y1+1) /\ y3 = 2*y1+1 /\ y1*y1   \leq x /\ x < y2
    /\ Safety_partialcorrectness

check ==
    /\ Safety_absence
    /\ Safety_partialcorrectness
    /\ Inv

=========
```

4 TD4

Exercice 9 *(malgtd1ex10,malgtd1ex10bis,malgtd1ex10ter,malgtd1ex10last)*
Pour montrer que chaque annotation est correcte ou incorrecte, on propose de procéder comme suit :

- *Traduire cette annotation sous la forme d'un contrat.*
- *Vérifier les conditions de vérification du contrat*

$\ell_1 : P_{\ell_1}(v)$
v := f(v, c)
$\ell_2 : P_{\ell_2}(v)$

variables v
requires $pre(v_0)$
ensures $post(v_0, v_f)$
begin
$\ell_1 : Q_1(v_0, v)$
v := f(v, c)
$\ell_2 : Q_2(v_0, v)$
end

- $pre(v_0) \equiv P_{\ell_1}(v_0)$
- $post(v_0, v_f) \equiv P_{\ell_2}(v_f)$.
- $Q_{\ell_1}(v_0, v) \equiv P_{\ell_1}(v) \wedge v = v_0$
- $Q_{\ell_2}(v_0, v) \equiv P_{\ell_2}(v)$

On rappelle qu'un contrat est valide si les trois conditions suivantes sont valides :

- *(init)* $pre(v_0) \wedge v = v_0 \Rightarrow Q_1(v_0, v)$
- *(concl)* $pre(v_0) \wedge Q_2(v_0, v) \Rightarrow post(v_0, v)$
- *(induct)* $pre(v_0) \wedge Q_1(v_0, v) \wedge cond_{\ell_1,\ell_2}(v) \wedge v' = f(v, c) \Rightarrow Q_2(v_0, v')$

Les deux propriétés (init) et (concl) sont valides par construction et la seule propriété à montrer correcte ou incorrecte est la propriété (induct).

Question 9.1 *(malgtd1ex10)*

$\ell_1 : x = 3 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
$y := z{+}x$
$\ell_2 : x = 3 \ \wedge \ y = x{+}6$

◇– **Solution de la question 9.1**
Modèle solution TLA$^+$:
../tlamodels/malgtd1ex10.tla

```
------------- MODULE malgtd1ex10 --------------------------------
EXTENDS Naturals, Integers, TLC, TLAPS
CONSTANTS x0,y0,z0
VARIABLES  x,y,z,pc
--------------------------------------------------------------------------------
(* Auxiliary definitions *)
typeInt(u) == u \in Int
pre(u,v,w) ==
    /\ u \in Int /\ v \in Int /\ w \in Int
    /\  u=3 /\ v=w+u /\ w=2*u

L == {"l1","l2"}

ppre == pre(x0,y0,z0)
--------------------------------------------------------------------------------
(* Interpretation: we assume that the precondition can hold and we have to find pos
ASSUME   ppre
-------------------------------------------------------------------------
(* Action for transition of the algorithm *)
al1l2 ==
```

```
    /\ pc="l1"
    /\ pc'="l2"
    /\ y'=z+x
    /\ z'=z /\ x'=x
-----------------------------------------------------------------
(* Computations *)
vars ==  <<x,y,z,pc>>
Next == al1l2  \/ UNCHANGED vars
Init == pc="l0" /\ x=x0 /\ y =y0 /\ z = z0  /\ pre(x0,y0,z0)
----------------------
(* Checking the annotation by checking the invariant i derived from the annotation
i ==
    /\ typeInt(x) /\ typeInt(y) /\ typeInt(z) /\ pc \in  L
    /\ pc="l1" =>  x=x0 /\ y=y0 /\ z=z0 /\ pre(x0,y0,z0)
    /\ pc="l2" =>  x=3 /\ y = x +6 /\ pre(x0,y0,z0)

Safe ==  i


Spec == Init /\ [][Next]_vars

==============================================================================
```

Fin 9.1

On définit un contrat comme suit :

variables x, y, z
requires $x0 = 3 \wedge y0 = z0{+}x0 \wedge z0 = 2.x0$
ensures $x_f = 3 \wedge y_f = x_f{+}6$
begin
$\ell_1 : x = 3 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
$y := z{+}x$
$\ell_2 : x = 3 \ \wedge \ y = x{+}6$
end

On pose les assertions suivantes à partir de l'annotation :

- $pre(x_0, y_0, z_0) \stackrel{def}{=} x0 = 3 \wedge y0 = z0{+}x0 \wedge z0 = 2.x0$
- $prepost(x_0, y_0, z_0, x, y, z) \stackrel{def}{=} x = 3 \ \wedge \ y = x{+}6$
- $Q_1(x_0, y_0, z_0, x, y, z) \stackrel{def}{=} x = 3 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
- $Q_2(x_0, y_0, z_0, x, y, z) \stackrel{def}{=} x = 3 \ \wedge \ y = x{+}6$

On établit les trois cpnditions pour valider le contrat :

- ***(init)*** $pre(x_0, y_0, z_0) \wedge (x, y, z) = (x_0, y_0, z_0) \Rightarrow Q_1(x_0, y_0, z_0, x, y, z)$
- ***(concl)*** $pre(v_0) \wedge Q_2(x_0, y_0, z_0, x, y, z) \Rightarrow prepost(x_0, y_0, z_0, x, y, z)$
- ***(induct)*** $pre(x_0, y_0, z_0) \wedge Q_1(x_0, y_0, z_0, x, y, z) \wedge TRUE \wedge (x', y', z') = (x, z{+}x, z) \Rightarrow Q_2(x_0, y_0, z_0, x', y', z')$
- $pre(x_0, y_0, z_0) \wedge Q_1(x_0, y_0, z_0, x, y, z) \wedge TRUE \wedge (x', y', z') = (x, z{+}x, z) \Rightarrow Q_2(x_0, y_0, z_0, x', y', z')$
- $\vdash pre(x_0, y_0, z_0) \wedge Q_1(x_0, y_0, z_0, x, y, z) \wedge TRUE \wedge (x', y', z') = (x, z{+}x, z) \Rightarrow Q_2(x_0, y_0, z_0, x', y', z')$
- $pre(x_0, y_0, z_0) \wedge Q_1(x_0, y_0, z_0, x, y, z) \wedge TRUE \wedge (x', y', z') = (x, z{+}x, z) \vdash Q_2(x_0, y_0, z_0, x', y', z')$
- $pre(x_0, y_0, z_0), Q_1(x_0, y_0, z_0, x, y, z), TRUE, (x', y', z') = (x, z{+}x, z) \vdash Q_2(x_0, y_0, z_0, x', y', z')$
- $x0 = 3 \wedge y0 = z0{+}x0, z0 = 2.x0, Q_1(x_0, y_0, z_0, x, y, z), TRUE, (x', y', z') = (x, z{+}x, z) \vdash Q_2(x_0, y_0, z_0, x', y', z')$
- $x0 = 3 \wedge y0 = z0{+}x0, z0 = 2.x0, x = 3 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash Q_2(x_0, y_0, z_0, x', y', z')$
- $x0 = 3 \wedge y0 = z0{+}x0, z0 = 2.x0, x = 3 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash x' = 3 \ \wedge \ y' = x'{+}6$
- $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash x' = 3 \ \wedge \ y' = x'{+}6$

- $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash x' = 3$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash x' = 3$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash x = 3$
  - $x = 3$ *est une hypoth'ese à gauche. Le séquent est valide.*
- $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash y' = x'{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash y' = x'{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash y' = x{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash z{+}x = x{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash 2.x{+}x = x{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash 2.3{+}3 = 3{+}6$
  - $x0 = 3, y0 = z0{+}x0, z0 = 2.x0, x = 3, y = z{+}x, z = 2{\cdot}x, TRUE, (x', y', z') = (x, z{+}x, z) \vdash 9 = 9$
  - *Réflexivité de l'égalité.*

Question 9.2 *(malgtd1ex10bis)*
Pour les deux exemples qui suivent, on considère dex cas et on doit donner une interprétation.

$\ell_1 : x = 2^4 \ \wedge\ y = 2 \ \wedge\ x{\cdot}y = 2^6$
$x := y{+}x{+}2^x$
$\ell_2 : x = 2^{10} \ \wedge\ y = 2$

$\ell_1 : x = 2^4 \ \wedge\ y = 2 \ \wedge\ x{\cdot}y = 2^5$
$x := y{+}x{+}2^x$
$\ell_2 : x = 2^{10} \ \wedge\ y = 2$

◇– **Solution de la question 9.2**
Modèle solution TLA$^+$:
../tlamodels/malgtd1ex10bis.tla

```
-------------------------------- MODULE malgtd1ex10bis ----------------------------

EXTENDS Naturals, Integers,TLC
CONSTANTS x0,y0
VARIABLES  x,y,pc
------------------------------------------------------------------------------
(* Interpretation: w assume that the precondition can hold and we have to find poss
ASSUME /\ x0 \in Int /\ y0 \in Int
       /\  x0=2^4 /\ y0=2 /\ x0*y0=2^5
-------------------------------------------------------------------------------
(* Auxiliary definitions *)
typeInt(u) == u \in Int
pre == /\ x0 \in Int /\ y0 \in Int
       /\  x0=2^4 /\ y0=2 /\ x0*y0=2^5
-------------------------------------------------------------------
```

```
(* Action for transitioon of the algorithm *)
all12 ==
    /\ pc="l1"
    /\ pc'="l2"
    /\ x'=y+x+2^x
    /\ y'=y
----------------------------------------------------------------
(* Computations *)
Next == all12  \/ UNCHANGED <<x,y,pc>>
Init == pc="l1" /\ x=x0 /\ y =y0  /\ pre
----------------------
(* Checking the annotation by checking the invariant i derived from the annotation
i ==
    /\ typeInt(x) /\ typeInt(y)
    /\ pc="l1" =>  x=x0 /\ y=y0 /\ pre
    /\ pc="l2" =>  x=2^10 /\ y = 2 /\ PrintT(x)

safe ==  i

=============================================================================
\* Modification History
\* Last modified Tue Feb 07 11:35:34 CET 2023 by mery
\* Created Wed Sep 09 18:19:08 CEST 2015 by mery
```

Fin 9.2

Question 9.3 *(malgtd1ex10ter.tla)*

$\ell_1 : x = 1 \ \wedge \ y = 12$
$x := 2{\cdot}y{+}x$
$\ell_2 : x = 1 \ \wedge \ y = 25$

Solution de la question 9.3

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex10ter.tla

```
------------------------------- MODULE malgtd1ex10ter ----------------------------

EXTENDS Naturals, Integers,TLC
CONSTANTS x0,y0
VARIABLES  x,y,pc
--------------------------------------------------------------------------------
(* Interpretation: w assume that the precondition can hold and we have to find poss
ASSUME /\ x0 \in Int /\ y0 \in Int
       /\  x0=1 /\ y0=12
-------------------------------------------------------------------------------
(* Auxiliary definitions *)
typeInt(u) == u \in Int
pre == /\ x0 \in Int /\ y0 \in Int
       /\  x0=1 /\ y0=12
-------------------------------------------------------------------
(* Action for transitioon of the algorithm *)
all12 ==
    /\ pc="l1"
    /\ pc'="l2"
```

```
    /\ x'=2*y+x
    /\ y'=y
-------------------------------------------------------------
(* Computations *)
Next == al1l2  \/ UNCHANGED <<x,y,pc>>
Init == pc="l1" /\ x=x0 /\ y =y0  /\ pre
----------------------
(* Checking the annotation by checking the invariant i derived from the annotation
i ==
    /\ typeInt(x) /\ typeInt(y)
    /\ pc="l1" =>  x=x0 /\ y=y0 /\ pre
    /\ pc="l2" =>  x=25 /\ y =  y0 /\ PrintT(x)

safe ==  i

=============================================================================
\* Modification History
\* Last modified Wed Mar 06 10:41:19 CET 2024 by mery
\* Created Wed Sep 09 18:19:08 CEST 2015 by mery
```

Fin 9.3

Question 9.4 *(malgtd1ex10last.tla)*

$\ell_1 : x = 11 \ \wedge \ y = 13$
$z := x; x := y; y := z;$
$\ell_2 : x = 26/2 \ \wedge \ y = 33/3$

◇– **Solution de la question 9.4**

Modèle solution TLA$^+$:
../tlamodels/malgtd1ex10last.tla

```
------------------------------- MODULE malgtd1ex10last ----------------------------

EXTENDS Naturals, Integers,TLC
CONSTANTS x0,y0,z0,UND
VARIABLES  x,y,z,pc

--------------------------------------------------------------------------------
(* Auxiliary definitions *)
typeInt(u) == u \in Int
pre == /\ x0 \in Int /\ y0 \in Int
       /\  x0=11 /\ y0=13 /\ z0 = UND
  ------------------------------------------------------------------------------
(* Interpretation: w assume that the precondition can hold and we have to find poss
ASSUME  pre

-------------------------------------------------------------------
(* Action for transitioon of the algorithm *)
al1l2 ==
    /\ pc="l1"
    /\ pc'="l2"
    /\ z'=x
    /\ x' = z
    /\ y' = z'
```

```
-----------------------------------------------------------------
(* Computations *)
Next == al1l2  \/ UNCHANGED <<x,y,z,pc>>
Init == pc="l1" /\ x=x0 /\ y =y0  /\ z = z0 /\ pre
----------------------
(* Checking the annotation by checking the invariant i derived from the annotation
i ==
    /\ pc="l1" =>  x=x0 /\ y=y0 /\ pre
    /\ pc="l2" =>  x=26 \div 2  /\ y = 33 \div 3

safe ==  i

==============================================================================
\* Modification History
\* Last modified Wed Feb 23 08:31:14 CET 2022 by mery
\* Created Wed Sep 09 18:19:08 CEST 2015 by mery
```

Fin 9.4

Exercice 10 *Montrer que chaque annotation est correcte ou incorrecte selon les conditions de vérifications énoncées comme suit*
$\forall x, y, , x', y'.P_\ell(x,y) \ \wedge \ cond_{\ell,\ell'}(x,y) \ \wedge \ (x',y') = f_{\ell,\ell'}(x,y) \Rightarrow P_{\ell'}(x',y')$

- $\ell_1 : x = 10 \ \wedge \ y = z{+}x \ \wedge z = 2{\cdot}x$
  $y := z{+}x$
  $\ell_2 : x = 10 \ \wedge \ y = x{+}2{\cdot}10$
- *On suppose que p est un nombre premier :*
  $\ell_1 : x = 2^p \ \wedge \ y = 2^{p+1} \ \wedge \ x{\cdot}y = 2^{2\cdot p+1}$
  $x := y{+}x{+}2^x$
  $anno\ell_2 : x = 5{\cdot}2^p \ \wedge \ y = 2^{p+1}$
- $\ell_1 : x = 1 \ \wedge \ y = 12$
  $x := 2{\cdot}y$
  $\ell_2 : x = 1 \ \wedge \ y = 24$
- $\ell_1 : x = 11 \ \wedge \ y = 13$
  $z := x; x := y; y := z;$
  $\ell_2 : x = 26/2 \ \wedge \ y = 33/3$

On rappelle qu'un contrat pour la correction partielle d'un petit programme est donné par les éléments ci-dessou en colonne de gauche et que les conditions de vérification associées sont définies par le texte de la colonne de droite.

Contrat de la correction partielle

variables $type\ X$

definitions
$def1 \stackrel{def}{=} text1$

requires $pre(x_0)$

ensures $post(x_0, x_f)$

begin
$0 : P_0(x_0, x)$
$\texttt{instruction}_0$
$1 : P_i(x_0, x)$
$\texttt{instruction}_1$
$f : P_f(x_0, x)$
end

Conditions de vérification
- $pre(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- $pre(x_0) \wedge P_f(x_0, x) \Rightarrow post(x_0, x)$
- Pour toutes les paires ℓ, ℓ', telles que $\ell \longrightarrow \ell'$, on vÃ©rifie que, pour toutes les valeurs $x, x' \in$ MEMORY
$$\left(\begin{array}{l} \left(\begin{array}{l} pre(x_0) \wedge P_\ell(x_0, x)) \\ \wedge cond_{\ell,\ell'}(x) \wedge x' = f_{\ell,\ell'}(x) \end{array} \right) \\ \Rightarrow P_{\ell'}(x_0, x') \end{array} \right),$$

Exercice 11 *(contrat1.c, pluscal_contrat1.tla)*
Soit le contrat suivant qui met en jeu les variables X,Y, Z,C,R.

VARIABLES $\texttt{int}\ X, Y, Z, C, R$

REQUIRES $x_0, y_0, z_0, c_0, r_0 \in \mathbb{Z}$

ENSURES $r_f = 0$

BEGIN
$0 : x = x_0 \wedge y = y_0 \wedge z = z_0 \wedge c = c_0 \wedge r = r_0 \wedge x_0, y_0, z_0, c_0, r_0 \in \mathbb{Z}$
$(X, Z, Y) := (49, 2{\cdot}C, (2{\cdot}C{+}1){\cdot}(2{\cdot}C{+}1));$
$1 : x = 49 \wedge z = 2{\cdot}c \wedge y = (z{+}1){\cdot}(z{+}1)$
$Y := X{+}Z{+}1;$
$2 : x = 49 \wedge z = 2{\cdot}c \wedge y = (c{+}1){\cdot}(c{+}1)$
END

Question 11.1 *Ecrire les conditions de vérification associée au contrat ci-dessus en vous aidant du rappel de la définition de ces conditions de vérification.*

Question 11.2 *Simplifier les conditions de vérification et préciser les conditions que doivent vérifier les valeurs initiales des variables X,Y,Z,C,R pour que les conditions de vérification soient toutes vraies. En particulier, il faudra s'assurer que la prÃ©condition est satisfaisable.*

Exercice 12 *()*

On considère le petit programme se trouvant à droite de cette colonne. Nous allons poser quelques questions visant à compléter les parties marquées en gras et visant à définir la relation de calcul.
On notera $pre(n_0, x_0, b_0)$ *l'expression* $n_0, x_0, b_0 \in \mathbb{Z}$ *et* $in(n, b, n_0, x_0, b_0)$ *l'expression* $n = n_0 \wedge b = b_0 \wedge pre(n_0, x_0, b_0)$

Question 12.1 *Donner l'assertion Requires en complétant ce qui est déjà mentionné et en reportant le texte complet de cette assertion Requires dans votre copie.*

On rappelle que la relation de transition de ℓ *vers* ℓ'*, notée* $a(\ell, \ell')$*, est définie par une relation de la forme* $cond_{\ell,\ell'}(v) \wedge v' = f_{\ell,\ell'}(v)$*.*

Question 12.2 *Ecrire les relations de transition entre les étiquettes successives :* $a(\ell_0, \ell_1)$*,* $a(\ell_1, \ell_2)$*,* $a(\ell_2, \ell_3)$*,* $a(\ell_3, \ell_6)$*,* $a(\ell_1, \ell_4)$*,* $a(\ell_4, \ell_5)$*,* $a(\ell_5, \ell_6)$*.*

VARIABLES $int\ N, X, B$

REQUIRES $n_0, x_0, b_0 \in \mathbb{Z}$

ENSURES $\left(\begin{array}{l} n_0 < b_0 \Rightarrow x_f = \textbf{question1} \\ n_0 \geq b_0 \Rightarrow x_f = \textbf{question1} \\ n_f = n_0 \wedge b_f = b_0 \end{array} \right)$

```
BEGIN
ℓ0 :
   X := N;
ℓ1 :
IF X < B THEN
   ℓ2 :
X := X·X+2·B·X+B·B;
   ℓ3 :
ELSE
   ℓ4 :
      X := B;
   ℓ5 :
FI
ℓ6 :
END
```

Fin de la série AA